In [4]:
```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
data = fetch_california_housing()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
n = X_test.shape[0]
p = X_test.shape[1]
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
print("Adjusted R2:", adjusted_r2)
```

```
MSE: 0.5558915986952423
RMSE: 0.745581383012775
MAE: 0.5332001304956994
R2 Score: 0.5757877060324523
Adjusted R2: 0.5749637928613573
```

In [5]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
np.random.seed(42)
data = {
    'Hours_Studied': [1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9,
10, 10, 10],
    'Pass': [0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
}
df = pd.DataFrame(data)
df.to_csv("student_data.csv", index=False)
X = df[['Hours_Studied']]
y = df['Pass']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Predictions for test set:", y_pred)
print("Actual values:", y_test.values)
new_student = pd.DataFrame({'Hours_Studied': [6]})
prediction = model.predict(new_student)
print("\nPrediction (0=Fail, 1=Pass):", prediction[0])
probability = model.predict_proba(new_student)
print(f"Probability of failing: {probability[0][0]:.2f}")
print(f"Probability of passing: {probability[0][1]:.2f}")
```

```
Predictions for test set: [0 1 1 0]
Actual values: [0 1 1 0]

Prediction (0=Fail, 1=Pass): 1
Probability of failing: 0.14
Probability of passing: 0.86
```

In [6]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
df = pd.read_csv("student_data.csv")
X = df[['Hours_Studied']]
y = df['Pass']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
new_student = pd.DataFrame({
    'Hours_Studied': [6]
})
prediction = model.predict(new_student)
print(y_pred)
print("Prediction (0=Fail, 1=Pass):", prediction[0])
```

```
[0 1 1 0]
Prediction (0=Fail, 1=Pass): 1
```

In [7]:
```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import numpy as np
iris = load_iris()
X = iris.data
y = iris.target
rf = RandomForestClassifier(

    n_estimators=100,
    oob_score=True,
    random_state=42
)
rf.fit(X, y)
print("OOB Accuracy:", rf.oob_score_)
print("OOB Error:", 1 - rf.oob_score_)
importance = pd.DataFrame({
    "Feature": iris.feature_names,
    "Importance": rf.feature_importances_
})
importance = importance.sort_values(by="Importance", ascending=False)
print("\nFeature Importance:")
print(importance)
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
print("\nPrediction:", rf.predict(sample))
```

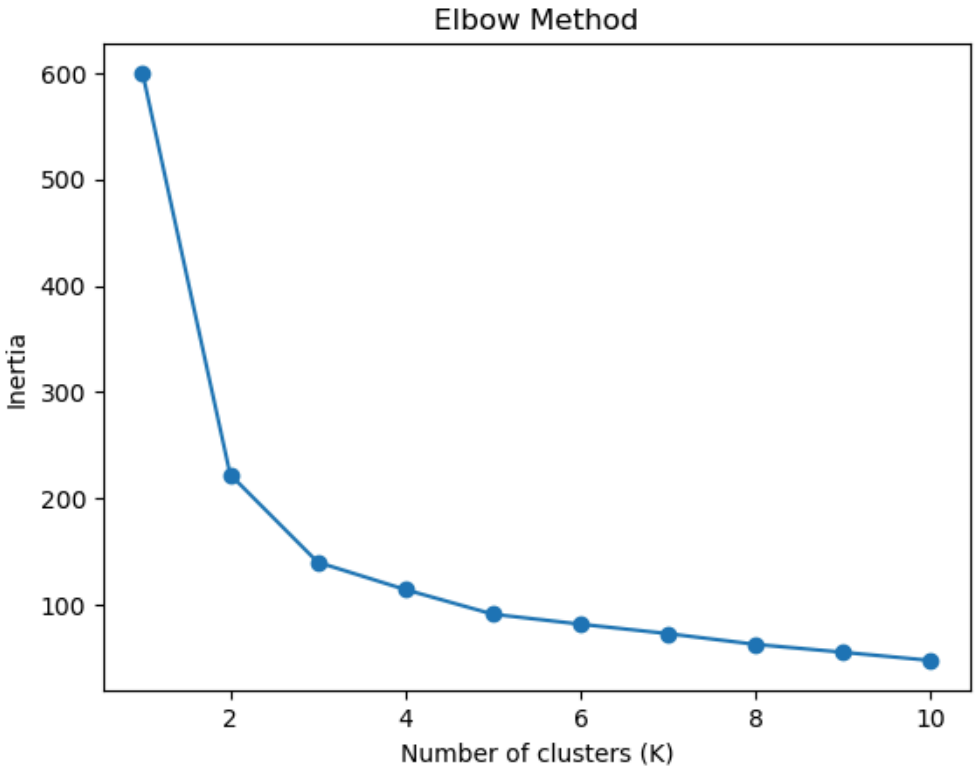
```
OOB Accuracy: 0.9533333333333334
OOB Error: 0.046666666666666634

Feature Importance:
             Feature  Importance
2  petal length (cm)    0.436130
3   petal width (cm)    0.436065
0  sepal length (cm)    0.106128
1   sepal width (cm)    0.021678

Prediction: [0]
```

In [8]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    roc_auc_score
)
import numpy as np
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
svm = SVC(
    kernel='rbf',
    probability=True,
    random_state=42
)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
y_prob = svm.predict_proba(X_test)
print("MODEL PERFORMANCE")
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)
precision = precision_score(y_test, y_pred, average='macro')
print("Precision:", precision)
recall = recall_score(y_test, y_pred, average='macro')
print("Recall:", recall)
f1 = f1_score(y_test, y_pred, average='macro')
print("F1 Score:", f1)
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)
roc = roc_auc_score(y_test, y_prob, multi_class='ovr')
print("\nROC-AUC Score:", roc)
sample = np.array([[5.1, 3.5, 1.4, 0.2]])
prediction = svm.predict(sample)
print("\nSingle Sample Prediction:", prediction)
```

```
MODEL PERFORMANCE
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Confusion Matrix:
 [[10  0  0]
```

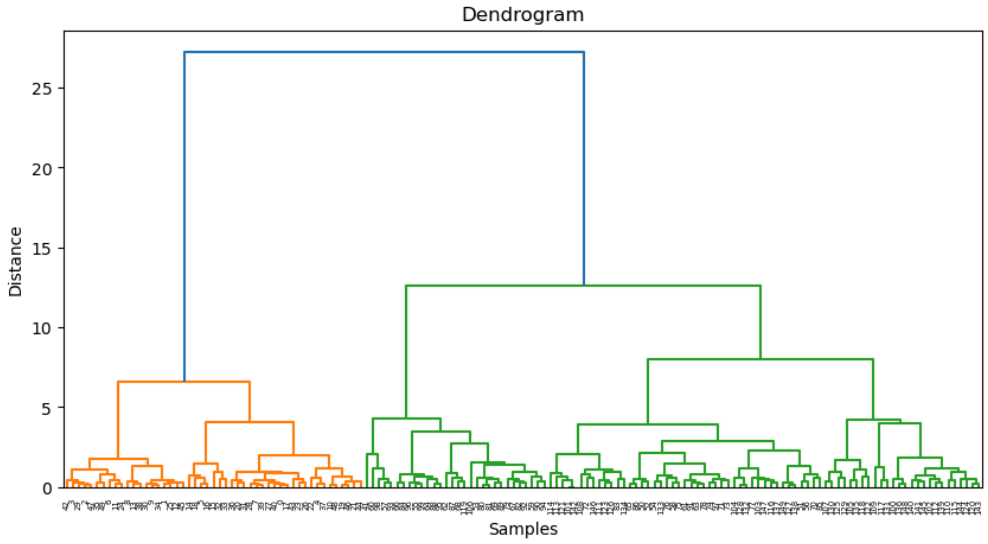
```
 [ 0  9  0]
 [ 0  0 11]]

ROC-AUC Score: 1.0

Single Sample Prediction: [0]
```

In [10]:
```
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
iris = load_iris()
X = iris.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
inertia = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
plt.plot(K_range, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.show()
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            s=300, c='red', marker='X')
plt.title("K-Means Clustering Result")
plt.show()
```



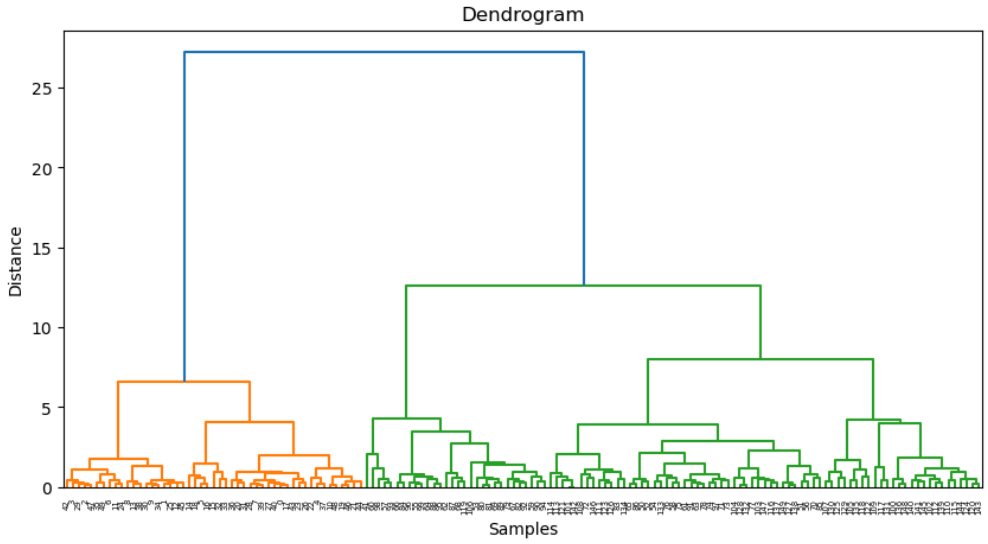

```
Silhouette Score: 0.45994823920518635
```

In [14]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
iris = load_iris()
X = iris.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
linked = linkage(X_scaled, method='ward')
plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()
hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)
labels = hc.fit_predict(X_scaled)
print("Cluster Labels:")
print(labels)
```




```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 2 1 1 1 1 1 1 1 0 0 0 2 0 2 0 2 0 2 2 0 2 0 2 0 2 2 2 2 0 0 0 0
 0 0 0 0 0 2 2 2 2 0 2 0 0 2 2 2 2 0 2 2 2 2 2 0 2 2 0 0 0 0 0 0 2 0 0 0 0
 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

In [13]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering
iris = load_iris()
X = iris.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
linked = linkage(X_scaled, method='ward')
plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.show()
hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)
labels = hc.fit_predict(X_scaled)
print("Cluster Labels:")
print(labels)
```




```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 2 1 1 1 1 1 1 1 0 0 0 2 0 2 0 2 0 2 2 0 2 0 2 0 2 2 2 2 0 0 0 0
 0 0 0 0 0 2 2 2 2 0 2 0 0 2 2 2 2 0 2 2 2 2 2 0 2 2 0 0 0 0 0 0 2 0 0 0 0
 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```

In [ ]: